

توثيق منظومة الـ Token Exchange للباك إند (.NET)

دليل مطوري الباك إند لربط وتكامل وثنائى Firebase Authentication مع مشروع EcoRide (Glider)

بسبب القيود المفروضة على خدمات إرسال رسائل الـ SMS والإيميلات التقليدية (مثل مشاكل الحدود والـ **Firebase** authorization في SendGrid/Twilio)، تم نقل منظومة التحقق والتسجيل بالكامل في الفلاتر ل تتم عبر **Authentication (Email/Password)** من ناحية تطبيق الموبايل (Client-Side).

لكي نقوم بربط وتأمين بقية العمليات الحيوية في التطبيق (مثل إدارة المحفظة، حجز السكوترز، والرحلات) دون مواجهة أخطاء **401 Unauthorized**، يجب تطبيق **آلية تبادل التوكن (Token Exchange Flow)** على سيرفر الـ الخاص بالنظام JWT حيث يقوم التطبيق بإرسال توكن الفايبريز الموثق، ويقوم السيرفر بالتحقق منه وإصدار الـ .NET.

💡 فكرة العمل باختصار:

المستخدم يسجل ويُفعّل إيميله في فايبريز → فلاتر يستخرج كود الـ **IdToken** المشفر → يُرسل التوكن إلى الـ .NET → سيرفر الـ .NET يتحقق من صحة التوكن عبر سيرفرات جوجل → يتم تسجيل/تحديث بيانات المستخدم في قاعدة البيانات المحلية → يُصدر الـ .NET الـ JWT الخاص بنا لتأمين بقية الـ APIs.

1 الخطوة الأولى: تهيئة Firebase Admin SDK في مشروع الـ .NET

لكي يتمكن سيرفر الـ .NET من فك تشفير والتحقق من التوكنات القادمة من الموبايل بشكل آمن، يجب تثبيت المكتبة الرسمية المعتمدة من جوجل وتميرير مفتاح الأمان الخاص بالمشروع:

تحميل حزمة NuGet:

افتح الـ **Package Manager Console** في مشروع الـ Web API ونفذ الأمر التالي:

```
Install-Package FirebaseAdmin
```

تحميل مفتاح الأمان (Private Key JSON):

1. توجه إلى لوحة تحكم **Firebase Console** الخاصة بالمشروع.
2. اضغط على أيقونة الترس (Project Settings) ثم اختر التبويب **Service accounts**.
3. اضغط على زر **Generate new private key**، سيتم تحميل ملف بصيغة **.json**.
4. قم بتضمين هذا الملف داخل مشروع الـ .NET في الفولدر الرئيسي بأمان (مثلاً باسم **firebase-config.json**).

⚠️ **تنبيه أمني:** لا تقم مطلقاً برفع ملف الـ JSON الخاص بمفاتيح السيرفر على مستودعات الأكواد العامة (مثل GitHub) لأنها تمنح صلاحيات كاملة لإدارة مشروع الفايبريز الخاص بك (GitHub).

2 الخطوة الثانية: كود ال DTO المتوقع استقباله من الفلاتر

تم إعداد الفلاتر ليدعم حالتين عند إرسال طلب المزامنة: إرسال بيانات عادية (JSON) في حالة تسجيل الدخول، أو إرسال طلب متعدد الأجزاء (Multipart/Form-Data) يتضمن صور البطاقة الشخصية والسيلفي في حالة تسجيل مستخدم جديد بالكامل. هذا هو الهيكل المطلوب لل DTO:

```
using Microsoft.AspNetCore.Http;

public class FirebaseSyncDto
{
    public string IdToken { get; set; } = string.Empty;
    public string? FullName { get; set; }
    public string? PhoneNumber { get; set; }

    // صور التحقق من الهوية (تأتي فقط في حالة التسجيل الجديد عبر ال Form-Data)
    public IFormFile? IdFrontPhoto { get; set; }
    public IFormFile? IdBackPhoto { get; set; }
    public IFormFile? SelfiePhoto { get; set; }
}
```

3 الخطوة الثالثة: تطبيق ال Endpoint داخل ال Controller

قم بإنشاء أو تحديث ال Endpoint المسمى `firebase-sync` داخل ال `AuthController` ليتعامل مع عملية فك التوكن واستبداله بال JWT المحلي:

```

using Microsoft.AspNetCore.Mvc;
using FirebaseAdmin.Auth;
using System.Threading.Tasks;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    // قم بحقن الـ Services الخاصة بك هنا (مثل IUserService, IJwtService)

    [HttpPost("firebase-sync")]
    [Consumes("application/json", "multipart/form-data")] // يدعم الطريقتين معاً
    public async Task<IActionResult> FirebaseSync([FromForm] FirebaseSyncDto dto)
    {
        try
        {
            // 1. التحقق من صحة التوكن وفكه عن طريق سيرفرات جوجل فايرباز
            FirebaseToken decodedToken = await
                FirebaseAuth.DefaultInstance.VerifyIdTokenAsync(dto.IdToken);
            string firebaseUid = decodedToken.Uid;
            string email = decodedToken.Claims["email"].ToString();

            // 2. التحقق من وجود المستخدم مسبقاً في قاعدة بياناتنا المحلية عن طريق الإيميل
            var user = await _userService.GetByEmailAsync(email);

            if (user == null)
            {
                // إذا كان المستخدم جديداً، نقوم بإنشائه وتخزين صور الهوية إن وجدت
                if (string.IsNullOrEmpty(dto.FullName))
                {
                    return BadRequest(new { Message = "بيانات التسجيل الكاملة مطلوبة"
                        ;({ "للمستخدمين الجدد."
                    }

                    // لوجيك إنشاء الحساب وتخزين الملفات المرفوعة (IdFrontPhoto, IdBackPhoto, SelfiePhoto)
                    user = await _userService.CreateUserFromFirebaseAsync(dto, firebaseUid,
                        email);
                }

                // 3. توليد الـ JWT Token الخاص بنظام الـ .NET (نفس لوجيك تسجيل الدخول القديم)
                var backendTokens = _jwtService.GenerateTokens(user);

                // 4. إرجاع النتيجة المتوافقة تماماً مع الـ Client في فلاتر
                return Ok(new
                {
                    User = user, // معرفتنا لدينا
                    Token = new
                    {
                        AccessToken = backendTokens.AccessToken,
                        RefreshToken = backendTokens.RefreshToken
                    }
                });
            }
            catch (FirebaseAuthException)
            {
            }
        }
    }
}

```

```

        return Unauthorized(new { Message = "توكن الفايريز غير صالح أو منتهي"
        };({ "الصلاحية."
        }
        catch (System.Exception ex)
        {
            return StatusCode(500, new { Message = "حدث خطأ داخلي في السيرفر", Details =
            ex.Message });
        }
    }
}

```

4 الخطوة الرابعة: ال Contract والمواصفات التقنية للطلب

المواصفات والتعريف	الخاصية
POST /api/Auth/firebase-sync	رابط ال API (Path)
multipart/form-data أو application/json	نوع المحتوى (Content-Type)
يُرد بـ JSON يحتوي على كائن ال user وال token الداخلي (Access & Refresh).	حالة التحقق الناجح (200 OK)
يُرد برفض الطلب إذا كان توكن الفايريز تم التلاعب به أو منتهي الصلاحية.	حالة التوكن المرفوض (401)

Example Successful JSON Response (200 OK):

```

{
  "user": {
    "id": "123",
    "email": "user@example.com",
    "fullName": "Mohamed Saied",
    "phoneNumber": "+201000000000"
  },
  "token": {
    "accessToken": "eyJhbGciOi...",
    "refreshToken": "d9b8c7..."
  }
}

```